



ENGINEERS LEVEL · GRADES 6-8

Curriculum Guide

4 Semesters + Capstone · Complete Teaching Document

TEACHER EDITION

Scope & sequence · lesson plans · theory · projects · quizzes · alignment · inclusive design

1. Scope & Sequence

Engineers is the capstone level of the K-8 journey: students move from JavaScript into **Python**, get **hands-on with AI**, and tackle **real cybersecurity and ethics**. Across five builds they write functions, train a model, design a responsible chatbot, analyze data, and prototype-and-pitch a solution to a real-world problem. The five CODEship strands reach their fullest expression here, with genuine **Technology & Society** depth — bias, privacy, and responsible AI.

Semester	Project	Coding focus	Big idea
1	Password Strength Checker	Python def, if/elif/else, len, strings; hashing concept	Functions decide — and real safety online
2	Smart Sorting AI	Train/label/classify a model; confidence; bias	Machines learn from data — and can be biased
3	Chatbot for Good	Dictionaries, input, intents; prompt design; ethics	AI can help — if designed responsibly
4	Data Detective	Lists, loops, mean/max/min; visualization	Find the story the data supports
Capstone	Innovation Challenge	Full toolkit + AI + security/ethics; SDG prototype	Build and pitch something that matters

How each semester runs

Each semester is **4 weekly classes** building to **1 project**, with a **theory unit**, **4 worksheets**, a **project planner**, and **2 quizzes of 10 questions** (Quiz 1 formative; Quiz 2 summative) plus a warm answer key. The **capstone** completes the level and the K-8 pathway. Per level: 20 sessions · 5 projects · 8 quizzes · 4 theory units.

AI and cybersecurity are hands-on here, not abstract. Students train a real classifier, design a working chatbot, and run explicit bias, privacy, and security checks — always paired with ethics.

2. Inclusive Design Playbook

Universal Design for Learning (UDL) is built into every lesson, and five accommodation profiles overlay on top. Both extend the platform's existing challenge-tags for reading, focus, and confidence.

UDL baseline (all learners): *Representation* — concepts shown as Python, spoken, and diagrammed (data flows, model pipelines); run every example. *Engagement* — meaningful problems (safety, fairness, real data, global challenges) and student choice. *Expression* — demonstrate learning by coding, tracing, analyzing, or pitching — multiple valid paths.

Accommodation profiles (Grade 6-8 specifics): **ADHD/focus:** short build cycles; run-and-see feedback; a syntax checklist; clear milestones. **ASD:** consistent tooling and naming; explicit rubrics; predictable pipelines; optional pairing. **Dyslexia/dysgraphia:** starter code and autocomplete; read-aloud tracing; never penalize identifier spelling; diagrams over dense text. **Anxiety/low-confidence:** scaffolded starters; "debugging is the job"; low-stakes first wins before the pitch. **Gifted/2e:** an "Engineer Extra" each class (optimize, add a class to the model, harden security, handle edge cases); autonomy and real challenge.

These map to the platform's challenge-tags (reading-support, focus, confidence) plus tags for ASD-routine and gifted-extension. One core curriculum + a reusable accommodation layer delivers variances without multiplying documents.

3. Semester 1 — "Password Strength Checker"

Real-world hook: the first Python build doubles as real cybersecurity — what actually makes an account safe. **Coding focus:** Python **def**, **if/elif/else**, `len` and string checks; the concept of **hashing**. **Big question:** *Can a function judge password strength — and why does hashing matter?*

Theory unit

Python defines functions with `def` and uses **indentation** for the body; **if/elif/else** selects among tiers. A checker measures length and variety. Cybersecurity core: long, unique passwords resist guessing, and **hashing** stores a one-way scramble so sites never keep your real password.

Lesson plans

Class 1 — Python functions. Obj: define and call a Python function. Align: Math C3 (Gr 6–8) subprograms. Warm-up: JS vs Python syntax. Teach: `def` + indentation. Coding space: write a length function. Practice: trace + write. Reflect: why indentation? Profiles: dyslexia — starter. Assess: call a function.

Class 2 — If/elif/else. Obj: build strength tiers. Align: C3 nested conditionals. Warm-up: rank passwords. Teach: `elif` chains. Coding space: return strong/okay/weak. Practice: complete conditions. Reflect: order of tests matters. Profiles: ADHD — one branch at a time. Extra: add a symbol check.

Class 3 — Strong & safe. Obj: reason about real security. Align: Tech & Society cybersecurity. Warm-up: 'was your fake password ever reused?'. Teach: `length > complexity`; unique per site; passphrases. Coding space: score real examples. Practice: rate + rules. Reflect: password-manager idea. Profiles: anxiety — discuss, no judgment. Assess: two safety rules.

Class 4 — Plan + hashing. Obj: design checker; explain hashing. Align: Tech & Society. Warm-up: 'should a site see your password?'. Teach: one-way hashing (age-appropriate). Coding space: finalize the checker. Practice: plan sheet. Reflect: hashing in one sentence. Profiles: all — partner test. Assess: define hashing simply.

Project outline

Students build a password strength checker: input a password, rate it with a Python function and `if/elif/else`, and explain length/uniqueness plus what hashing does.

Success criteria: a working function, tiered conditions, real strength factors, and a clear hashing explanation. Extension: add a passphrase suggestion or symbol/number checks.

Rubric (score 1-4 each)

Uses a Python function (def)

Uses if/elif/else

Checks real strength factors

Explains hashing/safety

Ontario alignment

Math C3 Coding (Gr 6-8): subprograms, nested conditionals

Technology & Society: cybersecurity, data protection

Language: explaining technical ideas clearly

4. Semester 2 — "Smart Sorting AI"

Real-world hook: students train a real image classifier and confront how bias enters AI. **Coding focus:** **Model, training data, labels, classify/confidence,** and **bias**. **Big question:** *How do machines learn — and how does bias get in?*

Theory unit

You train a **model** with **labelled examples**; it classifies new inputs with a **confidence**. If the **training data** is unbalanced, the model is **biased**. The lesson pairs the mechanics of ML with the ethics of data — what data, whose data, and fairness.

Lesson plans

Class 1 — Training data & labels. Obj: explain how a model learns. Align: Tech & Society AI. Warm-up: 'how would you teach 'cat'?'. Teach: examples + labels. Coding space: collect labelled images. Practice: match + reason. Reflect: why more/varied data? Profiles: ASD — clear steps. Assess: define training data.

Class 2 — Classify & confidence. Obj: read model output. Align: Math data; Tech & Society. Warm-up: predict a guess. Teach: confidence scores. Coding space: test new inputs. Practice: interpret results. Reflect: what if it's 50/50? Profiles: anxiety — worked example. Extra: add a third class.

Class 3 — Spot the bias. Obj: detect and fix bias. Align: Tech & Society ethics. Warm-up: unbalanced-data story. Teach: bias from data. Coding space: unbalance then rebalance a set. Practice: identify + fix. Reflect: real-world harms. Profiles: all — discussion-led. Assess: name a bias and fix.

Class 4 — Plan my classifier. Obj: design a fair classifier. Align: Design; ethics. Warm-up: pick classes. Teach: balance + ethics check. Coding space: finalize training. Practice: plan sheet. Reflect: is it fair? Profiles: gifted — measure accuracy per class. Assess: state one ethics safeguard.

Project outline

Students train an image classifier (e.g., recyclables), test it on new images, and explain how it learns — identifying one bias risk and how balanced data reduces it. Success criteria: trained with labelled data, classifies new inputs, names a bias risk, explains learning. Extension: compare accuracy across classes and rebalance.

Rubric (score 1-4 each)

Trains with labelled data

Classifies new inputs

Identifies a bias risk

Explains how the model learns

Ontario alignment

Technology & Society: artificial intelligence, bias, ethics of data

Math: data literacy; interpreting results

Science & Tech (STEM): investigation and testing

5. Semester 3 — "Chatbot for Good"

Real-world hook: a working chatbot for a real need — designed to be honest, helpful, and privacy-respecting. **Coding focus:** Python **dictionaries, input, intents; prompt design; AI ethics/privacy.** **Big question:** *Can we design AI that genuinely helps and respects privacy?*

Theory unit

A simple chatbot maps **intents** to **responses** with a **dictionary**, reads **input**, and replies. **Prompt design** — clear, specific instructions — shapes AI output. Responsible design means honesty (it's a bot), genuine helpfulness, a safe fallback, and protecting **privacy**.

Lesson plans

Class 1 — Dictionaries. Obj: use key→value maps. Align: C3 data structures. Warm-up: phonebook analogy. Teach: { } lookups. Coding space: build a replies dict. Practice: read + add entries. Reflect: dict vs list? Profiles: dyslexia — starter dict. Assess: look up a value.

Class 2 — Intents & responses. Obj: map messages to intents. Align: Language; Tech & Society. Warm-up: classify sample messages. Teach: intent → response; safe fallback. Coding space: handle input. Practice: match. Reflect: sensitive messages → suggest a trusted adult. Profiles: ASD — explicit mapping. Assess: write a fallback.

Class 3 — Prompt design. Obj: write clear prompts. Align: Language; AI literacy. Warm-up: fix a vague prompt. Teach: audience/length/focus. Coding space: improve bot replies. Practice: rewrite prompts. Reflect: why specificity helps. Profiles: anxiety — templates. Extra: add tone control.

Class 4 — Plan for good. Obj: design a responsible bot. Align: Tech & Society ethics/privacy. Warm-up: 'what should a bot never ask?'. Teach: ethics/privacy checklist. Coding space: finalize intents. Practice: plan sheet. Reflect: honest + private? Profiles: all — partner review. Assess: one privacy safeguard.

Project outline

Students build a chatbot for a real need (study buddy, well-being check-in): intents mapped in a dictionary, input handled, and an ethics/privacy check passed. Success criteria: a dict of intents, input handling, genuine helpfulness, and ethics/

privacy addressed. Extension: add prompt-shaped responses or a graceful unknown-message handler.

Rubric (score 1-4 each)

Uses a dict of intents

Handles input

Designed to genuinely help

Addresses ethics/privacy

Ontario alignment

Math C3 Coding (Gr 6-8): data structures (dictionaries)

Technology & Society: responsible AI, privacy

Language: purpose, audience, clear communication

6. Semester 4 — "Data Detective"

Real-world hook: students become analysts — turning a real dataset into an evidence-based insight. **Coding focus:** Python **lists** and **loops**; stats (`sum`, `len`, **mean**, `max/min`); **visualization**. **Big question:** *What story does the data actually support?*

Theory unit

Load a **dataset** into a **list**, loop to compute stats, then **visualize** to reveal a **trend**. An **insight** is a claim the data supports — and correlation is not causation, so limitations matter.

Lesson plans

Class 1 — Lists & loops. Obj: process data with loops. Align: C3 loops; Math data. Warm-up: total by hand. Teach: `for x in data`. Coding space: sum a list. Practice: trace. Reflect: why loops scale. Profiles: ADHD — step-through. Assess: give a total.

Class 2 — Statistics. Obj: compute mean/max/min. Align: Math data & probability. Warm-up: estimate the average. Teach: `sum/len = mean`. Coding space: compute stats. Practice: by hand + code. Reflect: which stat answers the question? Profiles: anxiety — worked example. Extra: median.

Class 3 — Read a chart. Obj: interpret and critique. Align: Math data; Language critical thinking. Warm-up: describe a trend. Teach: correlation $\neq$ causation. Coding space: plot the data. Practice: interpret + critique. Reflect: what else could explain it? Profiles: ASD — labelled examples. Assess: state the trend + one caveat.

Class 4 — Plan my investigation. Obj: design an analysis. Align: STEM inquiry; Design. Warm-up: pick a question. Teach: question $\rightarrow$ stat $\rightarrow$ chart $\rightarrow$ insight. Coding space: finalize. Practice: plan sheet. Reflect: is the insight supported? Profiles: gifted — add a second variable. Assess: state your insight.

Project outline

Students analyze a real small dataset: load into a list, compute a statistic, make a visualization, and state one supported insight with a limitation. Success criteria: loads/loops data, computes a correct stat, visualizes it, states a supported insight. Extension: compare two groups or add a second chart.

Rubric (score 1-4 each)

Loads/loops a dataset

Computes a stat correctly

Makes a visualization

States a supported insight

Ontario alignment

Math C3 Coding (Gr 6-8): lists, loops; Strand D data & statistics

Science & Tech: data-driven investigation

Language: evidence-based reasoning

7. Capstone — "Innovation Challenge"

Real-world hook: the final build of the K-8 journey — prototype and pitch a tech solution to a real-world problem. **Coding focus:** The full toolkit (Python/JS + **AI** + **cybersecurity/ethics**) applied through design thinking to an SDG-style challenge. **Big question:** *Can I build and pitch technology that genuinely helps?*

Theory unit

Students pick a real (SDG-style) problem, design a solution with their **full toolkit**, build a **prototype**, run a **security and ethics check**, **iterate** from feedback, and **pitch** the **impact**. It is the culmination of everything from Explorers to Engineers.

Lesson plans

Class 1 — Choose a challenge. Obj: scope a real problem. Align: Tech & Society; Language. Warm-up: SDG brainstorm. Teach: problem + who benefits. Coding space: create the project. Practice: problem statement. Reflect: why it matters. Profiles: anxiety — challenge menu. Assess: name the beneficiary.

Class 2 — Design the solution. Obj: select tools, sketch the system. Align: C3 planning; Design. Warm-up: which tools fit? Teach: system/data-flow diagram. Coding space: scaffold. Practice: plan sheet. Reflect: simplest viable build. Profiles: ASD — template. Assess: list your tools.

Class 3 — Prototype & security. Obj: build core; check security/ethics. Align: C3 debugging; Tech & Society. Warm-up: pseudocode main. Teach: security/ethics checklist. Coding space: build + protect data. Practice: check. Reflect: is it fair and safe? Profiles: gifted — threat-model an edge case. Assess: one safeguard.

Class 4 — Pitch & present. Obj: test, iterate, pitch. Align: Language presentation; Design. Warm-up: peer test. Teach: problem → solution → impact. Coding space: fix + polish. Practice: graduation checklist. Reflect: what did feedback change? Profiles: all — rehearse with a partner. Assess: deliver the pitch.

Project outline

Graduation gate: prototype and pitch a tech solution to a community or global problem, integrating code, AI, and security/ethics, tested and presented. Passing (all four rubric criteria) completes Engineers and the K-8 pathway.

Rubric (score 1-4 each)

Solves a real (SDG) problem

Integrates code + AI + security/ethics

Working prototype (tested)

Pitches impact clearly

Ontario alignment

Math C3 Coding (Gr 6-8): integrate subprograms, data structures, debugging

Technology & Society: responsible AI, cybersecurity, ethics

Language & Design: pitching, iteration, systems design

8. Alignment & Compliance

Engineers is built on the pan-Canadian five-area spine and tagged first to **Ontario**. Coding maps to **Mathematics Strand C3 (Coding), Grades 6-8** — subprograms, nested and combined conditionals, data structures, and debugging efficient code. Data work maps to **Strand D (Data & Statistics)**. The AI, bias, privacy, and cybersecurity content maps to **Technology & Society** and to **Science & Technology** STEM-skills expectations. Because this is the oldest band, alignment claims should be checked especially carefully against current documents.

Compliance language (locked): always describe materials as "**aligned to / maps to / supports**" provincial expectations — **never** "endorsed by" or "approved by" a ministry. Validate each alignment cell against the current official document before any public claim.

Cross-cutting (level-wide)

UDL is baked into every lesson; the five accommodation profiles overlay as reusable one-pagers. Provincial crosswalks (BC ADST, Alberta, Quebec + French) map over this same core content. Lessons, projects, quizzes, and alignment tags export to the platform schema with the coding space set to Python (with JavaScript review) plus AI-tool and security modules for this level.

DREAM. CODE. ACHIEVE. — Engineers completes the journey: Python, real AI, and cybersecurity, ending with a solution worth pitching. From Explorers to Engineers, the coder is ready.